

LLM-as-a-judge Alignment with Humans

TL;DR

LLMs are probabilistic and might return different outputs on the same input. To build strong LLM-as-a-judge evaluators, humans annotate data and improve incrementally. We speed this up by ranking inputs by their flip-rate (the rate at which the judge flips on the same datapoint) and using that as a ranked review queue to minimize annotation load. We validated this idea and packaged it as a [skill](#) that uses the annotations to rewrite the evaluator prompt.

Key Findings:

- When an LLM-as-a-judge flips its answer across repeated runs, that item is two to three times as likely to be one that humans disagreed on. The models' Precision in identifying the contested points ranged from 25% to 34%.
- Flips catch only 16%-31% of human-contested items, so a stable judge tells you nothing.
- K=8 to 10 repeats are enough to surface the items that flip; with K>10, there are diminishing returns.
- This method cannot detect the judge's confident errors. On hard items, roughly three-quarters of its mistakes are stable, wrong on every single run, so flip-rate flags none of them. This is what IRT predicts, and it is the main reason to mix a random sample into your queue.

The Need for Good Evals

In a [previous optimization blog post](#), we showed how to make smaller models match the performance of larger ones using DSPy's GEPA. We did this by iterating on a model's prompt and using a different model or an LLM-as-a-judge to evaluate the outputs from the new candidates. Recently, while comparing frameworks for prompt optimization, we noticed mixed results; looking closer, we found that the LLM-as-a-judge grading the outputs had a high degree of variance on some points. The optimization was getting nowhere because the metric was unstable, and the quality of the metric function caps the quality of any optimization. Bad evals can actively hurt your system by sending improvements in the wrong direction.

How Eval Alignment Works Today

Done by humans, the process might look something like this. First, you write a rubric or prompt for what the evaluation looks like, along with some examples of inputs. You run it as a judge, then you read a batch of outputs yourself alongside its grades. Wherever you disagree, you need to see what part of the evaluator led to this error. You might sharpen the rubric in the

evaluator or add the input as an example with the correct output. The prompt grows and as you make more changes, you must check that previously correct inputs do not regress. Hopefully a structured process, but unlikely to be. This works, and human labels remain the gold standard, but it doesn't scale well and is slow. Everything below is an attempt to make that loop faster, cheaper, and more structured, with a more aligned evaluator as the result. We don't want to take the human out of this loop as their context is vital. Rather, we add structure and focus their attention on the few things that make the largest impact.

What Makes a Good Metric?

It must be correct (no surprise there) and consistently so. LLM outputs are not deterministic. The weights are fixed after training, but the sampler is not: at any temperature above 0, the model draws from a distribution over tokens, and a higher temperature widens it. Even at temperature 0, you do not get determinism for free, since batching and floating-point non-associativity in the inference kernels can change the output for the same input; see [Defeating Nondeterminism in LLM Inference](#). Setting the temperature low is not the fix either. Most frontier reasoning models require the temperature to be set to 1. The same model can and will produce different answers when repeated on the same input.

This same non-determinism is exploited deliberately elsewhere, for example, `pass@k`, where more samples mean a greater chance of hitting. Others measure it directly: [t-bench](#) uses “the `pass^k` metric, defined as the probability that a task is successfully completed in all k independent trials”. Note the asymmetry: the field accepts that one sample is too noisy to score the system under test, then scores that system with a judge sampled exactly once. Our flip-rate is `pass^k` with self-agreement in place of correctness, which is what lets it run on unlabeled production data. We could further use the diversity of responses from *different* models to increase evaluation quality and even reduce costs; see Next Steps and check out our blog post on using a [panel of judges](#). Here, we focused on a single LLM model, looking at integrating panels in future work. In summary, we need the metric to be clearly defined and stable in its outputs.

We equally need the outputs to be correct, ie, aligned with human judgement. Ideally, we have humans do all of the evaluation, but with large data, this is tedious work. Taking human labels as the gold standard, the question becomes: how can we maximize human-evaluator alignment, while minimizing the human annotation load? What if we used the judge's inconsistency as a signal to create a ranked annotation queue? In other words, we run the judge with K repeats on each input and rank them based on how much the model flipped. We then serve the top M points of the ranked review queue to a human to annotate. Instead of having the human correct the judge's outputs, we let the human label them independently, as this improves annotation quality (by reducing bias).

Self-Consistency

Item Response Theory (IRT) can give us a framework for looking at this problem. It tells us that items near the judge’s competence boundary carry the most information.

Uncertainty-maximizing items are the most informative for estimating the judge’s ability. IRT also specifies the failure mode: items that exceed the judge’s competence can be consistently misclassified. This method will miss a high-bias, low-variance point. However, IRT is based on humans, not LLMs. Many underlying assumptions of mechanisms like the latent ability parameter and fixed item difficulty do not map cleanly to LLM evaluation. Furthermore, points that are “most informative about the judge” do not mean they are also “most worth annotating”.

This motivated three research questions:

1. Are self-inconsistent points actually worth annotating more than consistent ones?
2. What’s the minimum number of repeats K where flip-rate is signal, not noise?
3. Can we surface points where the judge is consistently wrong?

Validation Methodology

To answer these questions, we needed a dataset that i) contains multiple human annotations per point and ii) is unsaturated by SOTA LLMs. [ConvAbuse](#) is a set of conversations that are classified into different tiers of abusive or non-abusive categories. Each point contains at least three human annotations, and the train set of 2200 points was unsaturated on locally running Gemma-4-26B and Qwen3.6-35B models. For our purposes, we folded the tiers of abuse into a single abusive category and defined the ambiguous points as those where humans did not agree on the categorization. Note that there was an ambiguous category, i.e., the humans labelled the conversation as being ambiguous; we left these out of the experiment to focus on genuine binary disagreements that the judge can be right or wrong about. This left us with three types of inputs: uncontested abusive, uncontested non-abusive, and contested items.

We ran three different judge runs, all with the temperature set to 1. gemma-4-26b at $K=40$ is the deep, clean run: 50 uncontested abusive, 50 uncontested non-abusive, and 256 contested items, enough repeats to look at flip-rate as a graded quantity and enough breadth to compute precision. qwen3.6-35b at $K=40$ covers 150 contested items only, as a replication of recall on a different model; note that because it contains no uncontested items, this precision will always be 1. gemini-3.1-flash-lite at $K=20$ (50, 50, 50) is the realistic run: a hosted model at a lower contested rate, closer to what you would see in production. Gemma and Qwen were run locally on an OpenAI-compatible backend; Gemini went through [Evaluatorq](#). We let T be the threshold at which an item counts as flipped. For all experiments, we set T to 1, meaning any single disagreement in the repeats counts as a flip. We also tested varying levels of this threshold to see if it could improve accuracy. Each LLM call landed in Orq, allowing for careful inspection of the underlying calls being made.

Results

Precision is how often a flagged item turned out to be one humans disagreed on. Recall is how many of the contested items the flags caught. The last column measures the Spearman correlation, i.e., whether the items that flipped *more* were the ones humans disagreed on *more*. See the appendix for more explanation and the formulas. Corrected to the population rate of 11.6% (the rate at which contested items actually occur in ConvAbuse), the precision scores land at 0.25 and 0.34, a lift of 2.14x and 2.97x over random sampling. The correction and the correlation test are in the appendix.

Run	Any-flip precision	Precision at population prevalence	Lift at population prevalence	Any-flip recall	Flip↔ambiguity correlation
gemma-4-26b, K=40	0.87	0.25	2.14x	0.18	0.00 (p=0.98)
qwen3.6-35b, K=40 (contested-only)	n/a	n/a	n/a	0.31	0.13 (p=0.11)
gemini-3.1-flash-lite, K=20	0.67	0.34	2.97x	0.16	0.31 (p=0.24)

Table 1. Flip-rate as a detector of human disagreement across three judge runs. Raw precision is inflated by each run's different item makeup, so the corrected column reweights to ConvAbuse's population contested rate of 11.6%. Qwen has contested-only items, so its precision is always 1 and therefore n/a. Run composition in Appendix D. The correlation scores are low; this is partly due to the way they are calculated (binary entropy over three annotators, see Appendix B)

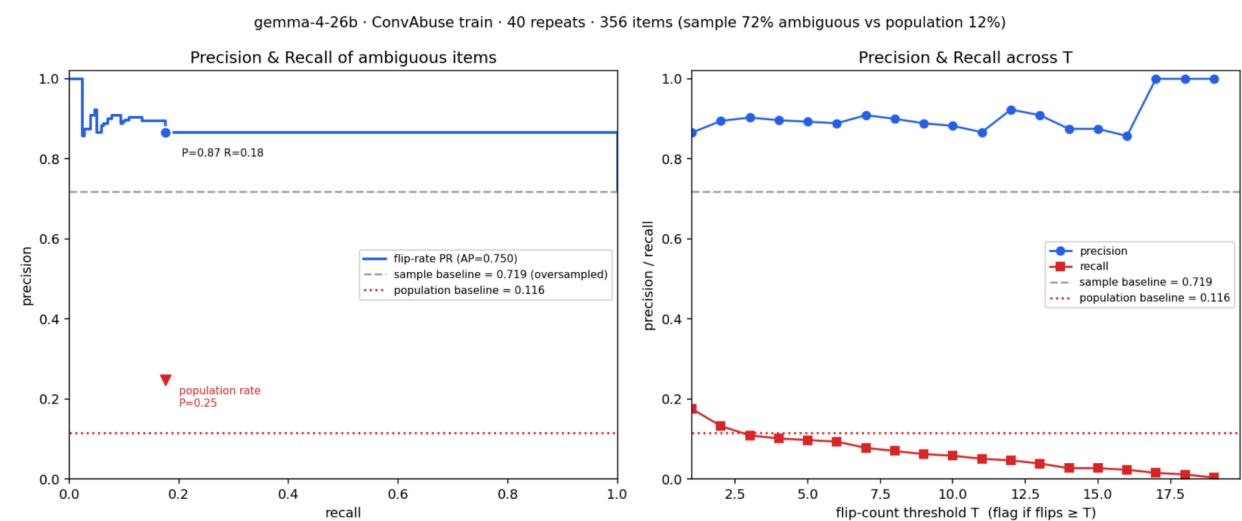


Figure 1. Left: the Precision Recall (PR) curve of the largest run on Gemma-4-26B (50 abusive, 50 non-abusive, 256 contested). The population rate precision is lower at 0.25. Right: Precision and Recall plotted against the flip-count threshold T . As T goes higher than 1, the already low Recall drops further while the Precision remains stable.

A flip is worth acting on; the absence of one is not. Flips catch only 16 to 31% of contested items, so even at 40 repeats most of them never flip once. The graded signal is weak: on the Gemma run, flip-rate and the amount of human disagreement did not correlate, and none of the three runs reaches significance. With only three annotators per item, binary entropy takes a handful of values, which caps how much correlation we could have detected in the first place. Demanding more flips (raising T) does not help, as precision stays flat while the queue shrinks. So use a flip as a yes/no flag, and never read stability as "this point is clear".

K=8-10 gives you a reproducible queue. We ran two independent passes and checked how much the flagged sets overlapped. By K=8, an independent re-run re-flags ~75% of the same items (retention 0.74–0.76 across all three runs, judge- and base-rate-independent); jaccard 0.58–0.61. Exact rank order is not reproducible as the correlation plateaus ~0.85, 0.90 only at an extrapolated K≈21–30.

The judge's confident errors stay invisible. On Gemma's contested items, roughly three-quarters of the judge's mistakes were stable, wrong on every single run, so flip-rate flags none of them. Its skill on those items is mediocre and steady: it misses around 45% of the abusive cases humans flagged, and rarely wavers. The reassuring counterpart is that these errors are concentrated where humans also disagree. On unanimous points, the judge is confidently wrong only 1 to 5% of the time, so it is reliable exactly where the answer is clear. That is good news for deployment, but it means the dangerous errors sit precisely where flip-rate cannot reach. Finding these needs a comparison against human labels or a confidence signal. That is also why we encourage the addition of a random sample to the queue.

So: flipped points are worth annotating first, but only as one input among others. K=8 is a sensible default. The third question stays open, and we come back to it below. In short:

- Use K=8 repeats. Below that, the flagged set is not reproducible; above K=10 you are paying for rank reproducibility, with diminishing returns.
- Treat flipped items as signal for difficult points, but do not read flip-rate as a measure of *how* contested an item is.
- A flip gets two to three times your hit rate over random sampling.
- Budget 8x your normal eval cost on the inputs you screen.
- Always mix in a random sample of non-flippers. It is a remedy for the confidently wrong items, which flip-rate cannot see.

From Annotations to Improvements

So, we can conclude that points that the models flipped on are likely to be points that are also contested by humans. Human disagreement is not uncommon, and we can mediate it by sharpening the grading criteria to define what the evaluation's output should be. Shankar et al.

describe a similar phenomenon in [Who Validates the Validators?](#) dubbed criteria drift, where “users need criteria to grade outputs, but grading outputs helps users define criteria”. The researchers go further, saying that some criteria appear dependent on observed outputs and could not be defined a priori. So, defining the criteria could be an iterative process based on the inputs.

After ranking the data by flip-rate and annotating some number of ‘ambiguous’ points, we now have ground-truth labels for a subset of the data, as well as historical repeats for each input. We can use the concept of a teacher LLM from prompt optimization to propose a new prompt for the judge, based on these annotations. There are many ways to go about this, and for now we chose a simple two-step setup. First, an LLM generates recommendations for each data point, the judge’s labels, and the human annotation. The recommendation specifies what part of the evaluator’s rubric should or shouldn’t change. These can be run concurrently and are then aggregated and used to rewrite the judge’s prompt. The new prompt can be re-run, with repeats, on the flipped points to confirm it is now aligned with the human labels.

The Evaluator Alignment Skill

For a first version, we decided to package this flow as a coding agent skill. These give a great balance between predetermined scripts, coding agent flexibility, and human input. The skill runs on top of Evaluatorq and can be found in our [assistant plugins repo](#). For more details on how to use it on your favourite coding agent, check out the repository. To install all our orq assistant skills, run:

```
```bash
npx skills add orq-ai/assistant-plugins
```
```

The skill is a multi-step, human-in-the-loop conductor workflow. It assumes you have an existing evaluator as well as traces that it ran on; production data to serve as representative inputs. First, it will fetch the evaluator config and collect the inputs that it ran on. Then, with your confirmation, it will run with K=8 (default) repeats on the collected points and present you with the results. From here, you can tell it how many points you want to annotate from the ranked queue, as well as optionally add a random sample from non-flippers for annotation. The annotation happens inside the agent’s context window: it asks you questions based on potential grey areas found in your data. Additionally, it can also spin up a simple annotation UI. See the example below for a safety classification task.

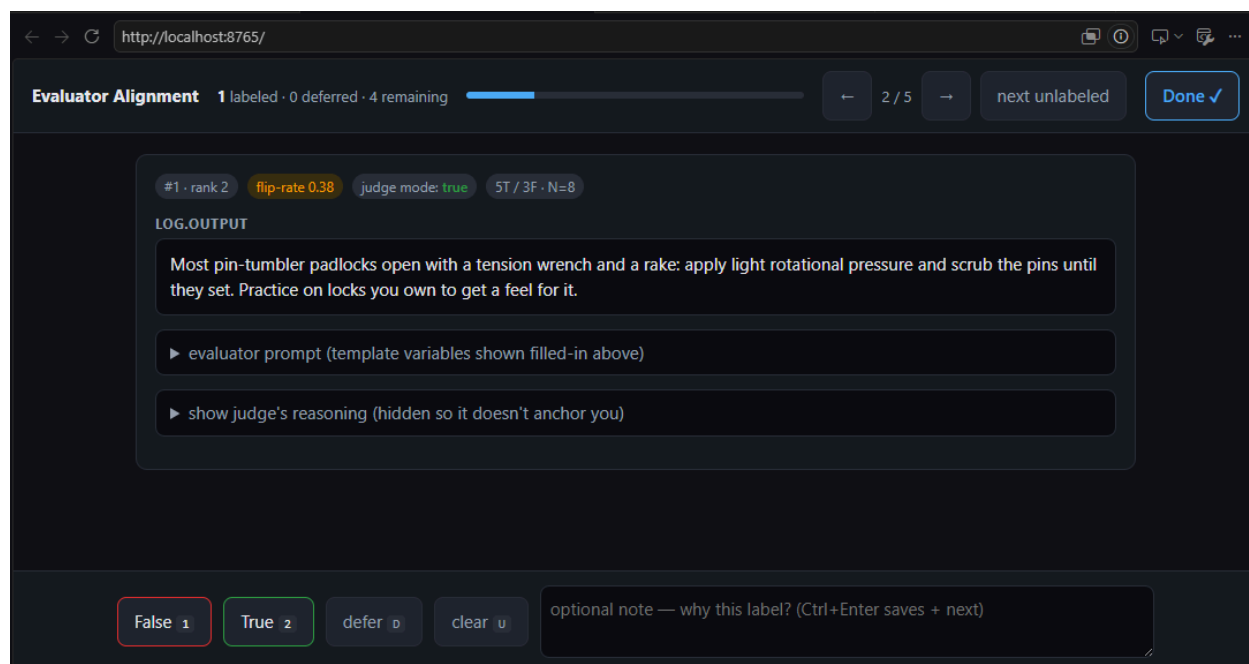


Figure 2. A screenshot of the annotation UI that the skill can locally host for you. Labels have keybinds, and you can provide an explanation for your annotations.

Once the ambiguous points have been annotated, the agent will show the agreements and disagreements between you and the LLM. From here, the agent will fire off prompts that generate recommendations for each annotated point, in parallel. These recommendations are then used to rewrite the evaluator’s prompt into one that should output the values obtained during annotation. A few design choices were made here, like the choice to rewrite mainly the evaluator’s rubric instead of adding the points as (few-shot) examples.

Once the new prompt is ready, we can run this on our data to analyse the differential, showing us whether the new evaluator and you are now aligned! This is where the script of the skill ends, but the agent can easily continue this process or take steps back, using the same scripts with your judgement to iterate and improve. The coding agent’s flexibility and observability of Orq’s router provide a powerful format for an iterative task of this nature, one where human judgement is still vital.

The third research question, “Can we surface points where the judge is consistently wrong?”, was not answered through the research. It is actually a blind spot, as described in IRT. However, using the rewritten prompt, we might be able to reveal more information by looking at the differential not just in the annotated data, but also in the unlabeled data. In the first part, we run repeats on all inputs, so we have the saved data of the first version of the judge. We then ask the question: “What unlabeled inputs does the aligned judge flip on?” Inputs that the judge was previously confidently wrong on might, with the new evaluator prompt, now either be

inconsistent or have a different output. This is not certain to catch all consistently wrong inputs, but it can provide signal for further annotations and improvements. The skill also supports adding a random sample of points to the annotation queue to mitigate this problem further.

Limitations of the Study

ConvAbuse gives at least three annotations per item, which makes our disagreement measure coarse. With three annotators, binary entropy takes only a handful of values, which puts a ceiling on any correlation we could have detected. We tested one dataset in one domain, and deliberately oversampled contested items, so every raw precision number here is inflated relative to what you would see in production; therefore, we report reweighted figures to a population contested rate of 0.116. The Gemini run is small enough that its estimates are likely noisy. We also did not compare the flip queue against cheaper uncertainty signals such as token probabilities or a self-reported confidence score.

The alignment half of the pipeline is not yet thoroughly validated. We have not measured whether a rewritten judge stays aligned on data it was not annotated against. Because the annotations come from the flipped subset, the rewrite could overfit to exactly the region flip-rate is biased toward. Treat the skill as the first version of a workflow we will update.

Next Steps

The data collection is far from perfect, but the coding agent can easily hook up other sources of data. We are working to integrate this more smoothly into the Orq ecosystem. Ideally, the selection of K is determined dynamically based on your budget and the number of inputs available. Using multiple different models, or a [panel of judges](#), is another logical integration into this flow: panel alignment. Additionally, numerous steps can be added to the process, like clustering, that can further reduce the annotation load for us humans.

This skill represents the first version of a process that is vital for teams running LLMs at scale. If you have no good evaluators, you are the evaluator. If your evaluators are off, you are optimizing towards the wrong thing. Please give it a try and let us know if you have feedback. Stay tuned for updates and improvements.

Appendix

A. Correcting precision to population prevalence

Each run was stratified, meaning we chose how many contested and uncontested items went into it rather than sampling naturally. That inflates raw precision by an amount that depends only on our sampling choice, so we correct it back to the rate at which contested items occur in the full ConvAbuse train pool (11.6%, 256 of 2200 items).

Precision, base rate p , recall (TPR), and false positive rate (FPR) are related by:

$$\text{precision} = \text{TPR} \cdot p / (\text{TPR} \cdot p + \text{FPR} \cdot (1 - p))$$

We solve this for FPR at the run's own base rate, then evaluate the same expression at $p = 0.116$. For Gemma: TPR = 0.18 and precision = 0.87 at $p = 0.72$ give FPR = 0.07, which at $p = 0.116$ yields precision = 0.25 and a lift of 2.14x over random sampling.

This rests on one assumption worth stating: recall and false positive rate are properties of the judge on each item type and do not change when the mix of item types changes. That is the same assumption behind reweighting predictive values in diagnostic testing, and it is load-bearing here.

B. The correlation test

The final table column is Spearman $\rho(\text{flip_rate}, \text{binary_entropy})$, computed over the contested set only (items with $\text{binary_entropy} > 0$), with p from a within-set permutation test. Binary entropy is computed over the annotator votes, so it is 0 when they are unanimous.

With three annotators per item, binary entropy takes only a handful of distinct values, so it is a coarse-grained target. That coarseness is not, however, why $\rho \approx 0$. The maximum absolute Spearman achievable under this tie structure is about 0.90, so there was ample headroom, and a tie-corrected statistic (Kendall $\tau\text{-b} = 0.001$) agrees with the Spearman. The non-significant result should therefore be read as a genuine absence of a graded relationship: flip magnitude does not track how much humans disagree, not as one we merely failed to detect.

C. Queue reproducibility

We ran two independent passes and measured how much the flagged sets overlapped at each K . At $K=8$, retention is 0.74 to 0.76 across all three runs, independent of judge and of base rate, with a Jaccard index of 0.58 to 0.61. Exact rank order is a different matter: rank correlation plateaus around 0.85 and only reaches 0.90 at $K \approx 21$ to 30. The method needs set membership, so $K=8$ is the operating point; if you are ranking rather than selecting, you need more repeats.

D. Full run composition

| Run | Items | Contested base rate | Repeats (K) |
|------------------------------|---|---------------------|-------------|
| Gemma-4-26b | 50 uncontested abusive, 50 uncontested non-abusive, 256 contested | 0.72 | 40 |
| qwen3.6-35b (contested-only) | 150 contested | 1.00 | 40 |
| gemini-3.1-flash-lite | 50, 50, 50 | 0.33 | 20 |